

Reviewer Pack — Minimal Order of Symmetric Braid Groups and Resolution Proof...

Entry 528d438fc7ec · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 04:42:13 UTC

Minimal Order of Symmetric Braid Groups and Resolution Proof Width Correlation

Entry ID: 528d438fc7ec **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 04:42:13 UTC

1. Conjecture

Field A (mathematical branch): Symmetric Braid Groups Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

The minimal order of the symmetric braid group (obtained by removing trivial braids) for a given instance φ of SAT is linearly correlated with its resolution proof width $w(\varphi)$, such that $\min_order(SB(\varphi)) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Symmetric braid groups are known to be related to the counting and complexity of combinatorial structures, which may have implications for the resolution proof complexity. This conjecture aims to explore this connection by proposing a quantitative relationship between the two.

Taxonomy category: braids (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ed847eecd153560

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $\min_order(SB(\varphi))$ and $w(\varphi)$ is ≥ 0.8 for all seeds, with no seed showing $|\min_order(SB(\varphi)) - w(\varphi)| > 3$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - symmetric braid groups AND resolution proof complexity - min_order SB(ϕ) AND $\theta(w(\phi))$ - SAT instance AND resolution proof width linear correlation

Top relevant hits considered: - [s2:10.1177/09544089251325921] Efficient microsink design to maximize the thermal performance - [s2:10.1016/J.PHYSLETB.2021.136146] First measurement of quarkonium polarization in nuclear collisions at the LHC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])
    if cols_A != rows_B:
        raise ValueError("Incompatible dimensions for matrix multiplication")
    result = [[sum(A[i][k] * B[k][j] for k in range(cols_A)) for j in range(cols_B)] for i in range(rows_A)]
    return result

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for col in range(cols):
        pivot_row = None
        for row in range(col, rows):
            if matrix[row][col] != 0:
                pivot_row = row
                break
        if pivot_row is None:
            continue
        matrix[col], matrix[pivot_row] = matrix[pivot_row], matrix[col]
```

```

    for i in range(rows):
        if i == col:
            continue
        factor = matrix[i][col] / matrix[col][col]
        for j in range(cols):
            matrix[i][j] -= factor * matrix[col][j]
    return matrix

def rank(matrix):
    rows, cols = len(matrix), len(matrix[0])
    augmented_matrix = [row[:] + [1 if i == j else 0 for j in range(rows)] for i, row in enumerate(matrix)]
    reduced_matrix = gaussian_elimination(augmented_matrix)
    rank = sum(1 for row in reduced_matrix if any(row[i] != 0 for i in range(cols)))
    return rank

def min_order_symmetric_braid_group(clauses):
    n_vars = max(max(abs(lit) for lit in clause) for clause in clauses)
    n_clauses = len(clauses)
    matrix = [[0] * (n_vars + n_clauses) for _ in range(n_vars)]
    for i, clause in enumerate(clauses):
        for lit in clause:
            if lit > 0:
                matrix[lit - 1][i + n_vars] = 1
            else:
                matrix[-lit - 1][i + n_vars] = -1
    return rank(matrix)

def resolution_proof_width(phi):
    # Placeholder function; actual implementation required
    return len(phi) # Simplified for demonstration

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for _ in range(30): # Ensure statistical robustness
        n = random.randint(5, 40)
        phi = [[random.randint(-n, n) for _ in range(random.randint(1, n))] for _ in range(n)]
        min_order = min_order_symmetric_braid_group(phi)
        width = resolution_proof_width(phi)
        results.append((min_order, width))
    metric_value = sum(abs(min_order - width) for min_order, width in results) / len(results)
    conjecture_holds = all(0 <= abs(min_order - width) <= 3 for min_order, width in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"
    return {
        "metric_name": "Mean Absolute Difference",
        "metric_value": metric_value,
        "instances_tested": len(results),
        "n_max": max(n for _ in range(30)),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(abs(min_order - width) > 3 for min_order, width in sum((r["instances_tested"], r["metric_value"]) for r in results)):
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={seeds[next(iter(seeds.values()))]}")
else:
    print(f"RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ion', 'metric_value': None, 'instances_tested': 0, 'n_max': 18, 'conjecture_holds': False, 'counterexample': None, 'width(72) > 3'}
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 15, 'conjecture_holds': False, 'counterexample': None, 'width(60) > 3'}
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 7, 'conjecture_holds': False, 'counterexample': None, 'width(28) > 3'}
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 25, 'conjecture_holds': False, 'counterexample': None, 'width(100) > 3'}
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 23, 'conjecture_holds': False, 'counterexample': None, 'width(92) > 3'}
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 9, 'conjecture_holds': False, 'counterexample': None, 'width(36) > 3'}
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 38, 'conjecture_holds': False, 'counterexample': None, 'width(152) > 3'}
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 16, 'conjecture_holds': False, 'counterexample': None, 'width(64) > 3'}
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 36, 'conjecture_holds': False, 'counterexample': None, 'width(144) > 3'}
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 19, 'conjecture_holds': False, 'counterexample': None, 'width(76) > 3'}
TRIAL: {'metric_name': 'correlation', 'metric_value': None, 'instances_tested': 0, 'n_max': 17, 'conjecture_holds': False, 'counterexample': None, 'width(68) > 3'}
RESULT: FALSIFIED counterexample="correlation_coefficient < 0.8 or abs(min_order - width) > 3" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code does not implement the resolution proof width calculation, which is a crucial part of the conjecture. The placeholder function `resolution_proof_width(phi)` suggests that this aspect has not been implemented, and thus the test cannot accurately measure

the correlation between the minimal order of symmetric braid groups and resolution proof width.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that the conjecture does not hold for at least one seed, as the correlation coefficient is less than 0.8 or the absolute dif | next: Implement the resolution proof width calculation and retest the conjecture to verify its validity.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13499
2	preregistration	ollama_remote	glm4:latest	0	0	12060
3	novelty	ollama_remote	glm4:latest	0	0	14392
4	novelty	ollama_remote	glm4:latest	0	0	9843
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13178
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16979
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11455
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19493
9	critic	ollama_remote	glm4:latest	0	0	19476
10	judge	ollama_remote	glm4:latest	0	0	20670

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 151045 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/528d438fc7ec.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/528d438fc7ec.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/528d438fc7ec.tar.gz (if generated)